

[Algorithm]

삽입 정렬 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

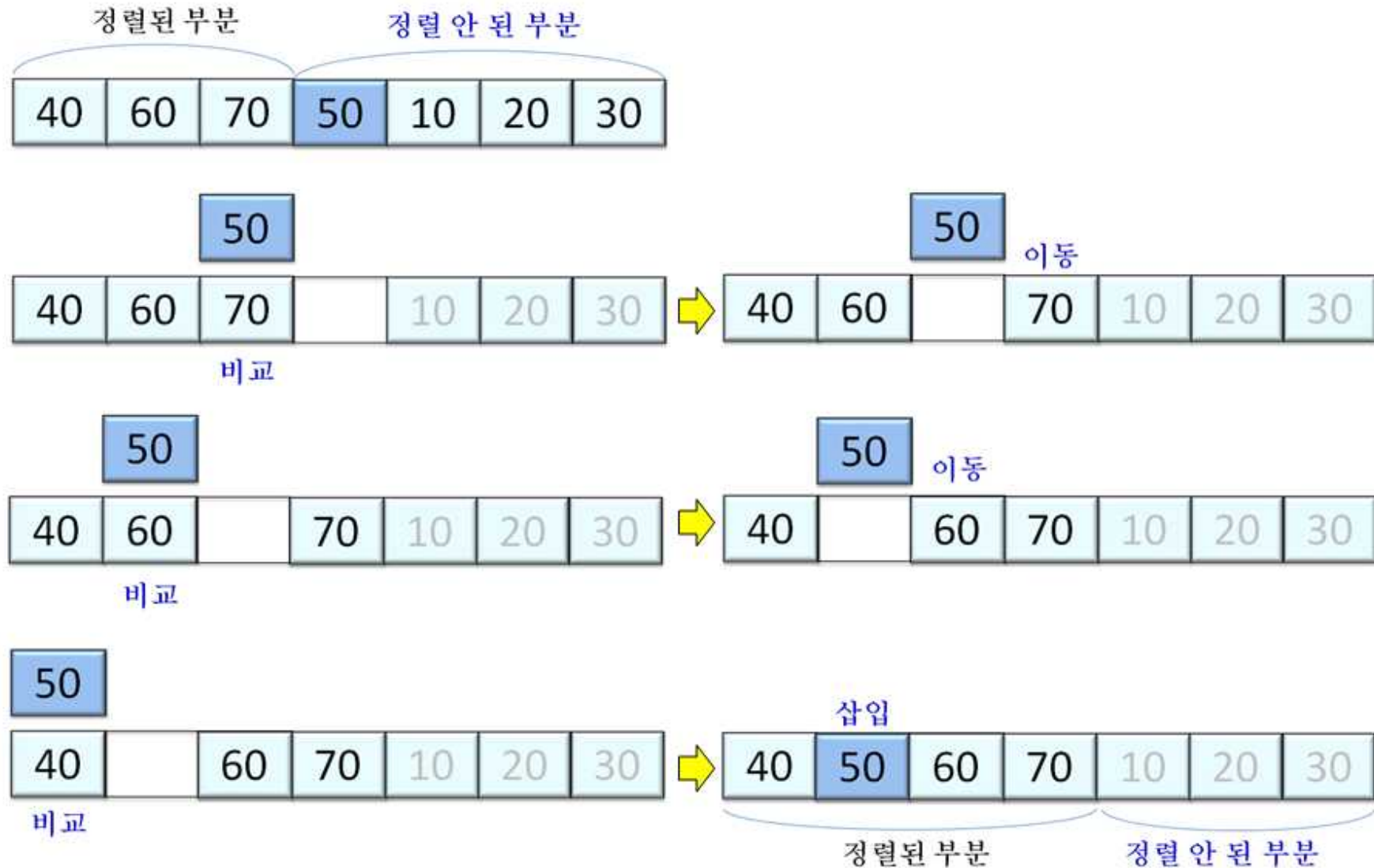
삽입 정렬

□ 삽입 정렬 (Insertion Sort)

- 배열을 정렬된 부분 (앞부분)과 정렬 안 된 부분 (뒷부분)으로 나누고, 정렬 안 된 부분의 가장 왼쪽 원소를 정렬된 부분의 적절한 위치에 삽입하여 정렬되도록 하는 과정을 반복한다.



삽입 정렬



삽입 정렬

- 정렬 안 된 부분의 숫자 하나가 정렬된 부분에 삽입됨으로써, 정렬된 부분의 원소 수가 1개 늘어나고, 동시에 정렬이 안 된 부분의 원소 수는 1개 줄어든다.
- 이를 반복하여 수행하면, 마지막엔 정렬이 안 된 부분엔 아무 원소도 남지 않고, 정렬된 부분에 모든 원소가 있게 된다.

InsertionSort 알고리즘

입력: 크기가 n 인 배열 A

출력: 정렬된 배열 A

1. for $i = 1$ to $n-1$ {
2. CurrentElement = $A[i]$ // 정렬 안된 부분의 가장 왼쪽원소
3. $j \leftarrow i - 1$ // 정렬된 부분의 가장 오른쪽 원소로부터 왼쪽
 방향으로 삽입할 곳을 탐색하기 위하여
4. while ($j \geq 0$) and ($A[j] > \text{CurrentElement}$) {
5. $A[j+1] = A[j]$ // 자리 이동
6. $j \leftarrow j - 1$
7. $A[j+1] \leftarrow \text{CurrentElement}$
8. }
9. } return A

InsertionSort 알고리즘

- 삽입 정렬은 정렬된 부분에는 $A[0]$ 만이 있는 상태에서 정렬이 시작된다.
- Line 1
 - CurrentElement의 배열 인덱스 i 를 위해 for-루프를 이용하여 $1 \sim (n-1)$ 까지 변한다.
- Line 2
 - 정렬 안 된 부분의 가장 왼쪽에 있는 원소인 $A[i]$ 를 CurrentElement로 놓는다.
- Line 3
 - ' $j=i-1$ '은 j 가 정렬된 부분의 가장 오른쪽 원소의 인덱스가 되어 왼쪽 방향으로 삽입할 곳을 탐색하기 위함이다.

InsertionSort 알고리즘

□ Line 4~6

- CurrentElement가 삽입될 곳을 찾는다.
- while-루프의 조건 ($j \geq 0$)은 배열 인덱스로 사용되는 j 가 배열의 범위를 벗어나는 것을 방지하기 위함이고,
- 2번째 조건 ($A[j] > \text{CurrentElement}$)는 $A[j]$ 가 CurrentElement보다 크면 $A[j]$ 를 오른쪽으로 1칸 이동시키기 위함이다.
- 이러한 자리이동은 line 5에서 수행된다.
- Line 6에서는 j 를 1 감소시켜 바로 왼쪽 원소에 대해 while-루프를 반복적으로 수행함

□ Line 7

- $A[j+1]$ 에 CurrentElement를 저장한다.
- 이는 while-루프가 끝난 직후에는 $A[j]$ 가 CurrentElement보다 크지 않으므로 $A[j]$ 의 오른쪽 (즉, $A[j+1]$)에 CurrentElement를 삽입해야 하기 때문이다.

삽입 정렬의 수행 과정

0	1	2	3	4	5	6	7
40	10	50	90	20	80	30	60

- $i=1$, $\text{CurrentElement}=A[1]=10$, $j = i-1 = 0$

0	1	2	3	4	5	6	7
40	10	50	90	20	80	30	60

$A[j]=A[0] > \text{CurrentElement}=10$ 이므로

0	1	2	3	4	5	6	7
	40	50	90	20	80	30	60

한 칸 앞으로 이동

$j=j-1=0-1=-1$ 이므로, $A[j+1]=A[-1+1]=A[0]$ 에 $\text{CurrentElement}=10$ 을 저장한다.

0	1	2	3	4	5	6	7
10	40	50	90	20	80	30	60

삽입 정렬의 수행 과정

- $i=2$, $\text{CurrentElement}=A[2]=50$, $j=i-1=1$

0	1	2	3	4	5	6	7
10	40	50	90	20	80	30	60

$A[j]=A[1] < \text{CurrentElement}=50$ 이므로, 자리이동 없이 50이 그 자리에 위치한다.

- $i=3$, $\text{CurrentElement}=A[3]=90$, $j=i-1=2$

0	1	2	3	4	5	6	7
10	40	50	90	20	80	30	60

$A[j]=A[2] < \text{CurrentElement}=90$ 이므로, 자리이동 없이 90이 그 자리에 위치한다.

- $i=4$, $\text{CurrentElement}=A[4]=20$, $j=i-1=3$

0	1	2	3	4	5	6	7
10	40	50	90	20	80	30	60

$A[j]=A[3] > \text{CurrentElement}=20$ 이므로

0	1	2	3	4	5	6	7
10	40	50		90	80	30	60

한 칸 앞으로 이동

$j=j-1=3-1=2$ 이고, $A[j]=A[2] > \text{CurrentElement}=20$ 이므로

0	1	2	3	4	5	6	7
10	40		50	90	80	30	60

한 칸 앞으로 이동

$j=j-1=2-1=1$ 이고, $A[j]=A[1] > \text{CurrentElement}=20$ 이므로

0	1	2	3	4	5	6	7
10		40	50	90	80	30	60

한 칸 앞으로 이동

$j=j-1=1-1=0$ 이고, $A[j]=A[0] < \text{CurrentElement}=20$ 이므로, $A[j+1] = A[0+1] = A[1]$ 에 $\text{CurrentElement}=20$ 을 저장한다.

0	1	2	3	4	5	6	7
10	20	40	50	90	80	30	60

삽입 정렬의 수행 과정

- $i=5$, $\text{CurrentElement}=A[5]=80$ 일 때의 결과

0	1	2	3	4	5	6	7
10	20	40	50	80	90	30	60

- $i=6$, $\text{CurrentElement}=A[6]=30$ 일 때의 결과

0	1	2	3	4	5	6	7
10	20	30	40	50	80	90	60

- $i=7$, $\text{CurrentElement}=A[7]=60$ 일 때의 결과

0	1	2	3	4	5	6	7
10	20	30	40	50	60	80	90

시간복잡도

- 삽입 정렬은 line 1의 for-루프가 (n-1)번 수행되는데,
 - i=1일 때 while-루프는 1번 수행되고,
 - i=2일 때 최대 2번 수행되고, ...,
 - 마지막으로 최대 (n-1)번 수행되므로
- 루프 내부의 line 5~6이 수행되는 총 횟수:
$$1 + 2 + 3 + \dots + (n-2) + (n-1) = n(n-1)/2$$
- 루프 내부의 수행시간은 $O(1)$ 이므로,
- 시간복잡도: $n(n-1)/2 \times O(1) = O(n^2)$

시간복잡도

- 삽입 정렬은 입력의 상태에 따라 수행 시간이 달라질 수 있다.
- 입력이 이미 정렬되어 있으면, 항상 각각 CurrentElement가 자신의 왼쪽 원소와 비교 후 자리 이동 없이 원래 자리에 있게 되고, while-루프의 조건이 항상 '거짓'이 되므로 원소의 이동도 없다.
- 따라서 $(n-1)$ 번의 비교만 하면 정렬을 마치게 된다.
- 이때가 삽입 정렬의 최선 경우이고 시간복잡도는 $O(n)$ 이다.
- 삽입 정렬은 거의 정렬된 입력에 대해서 다른 정렬 알고리즘보다 빠르다.

삽입 정렬 알고리즘 구현1

입력: 리스트 a
출력: 정렬된 새 리스트
리스트 r에서 v가 들어가야 할 위치를 돌려주는 함수

```
def find_ins_idx(r, v):  
    # 이미 정렬된 리스트 r의 자료를 앞에서부터 차례로 확인하여  
    for i in range(0, len(r)):  
        # v 값보다 i번 위치에 있는 자료 값이 크면  
        # v가 그 값 바로 앞에 놓여야 정렬 순서가 유지됨  
        if v < r[i]:  
            return i  
    # 적절한 위치를 못 찾았을 때는  
    # v가 r의 모든 자료보다 크다는 뜻이므로 맨 뒤에 삽입  
    return len(r)
```

```
def ins_sort(a):  
    result = [] # 새 리스트를 만들어 정렬된 값을 저장  
    while a: # 기존 리스트에 값이 남아 있는 동안 반복  
        value = a.pop(0) # 기존 리스트에서 한 개를 꺼냄  
        ins_idx = find_ins_idx(result, value) # 꺼낸 값이 들어갈 적당한 위치 찾기  
        result.insert(ins_idx, value) # 찾은 위치에 값 삽입(이후 값은 한 칸씩 밀려남)  
    return result
```

```
d = [2, 4, 5, 1, 3]  
print(ins_sort(d))
```

삽입 정렬 알고리즘 구현2

삽입 정렬
입력: 리스트 a
출력: 없음(입력으로 주어진 a가 정렬됨)

```
def ins_sort(a):  
    n = len(a)  
    for i in range(1, n): # 1부터 n-1까지  
        # i번 위치의 값을 key로 저장  
        key = a[i]  
        # j를 i 바로 왼쪽 위치로 저장  
        j = i - 1  
        # 리스트의 j번 위치에 있는 값과 key를 비교해 key가 삽입될 적절한 위치를 찾음  
        while j >= 0 and a[j] > key:  
            a[j + 1] = a[j] # 삽입할 공간이 생기도록 값을 오른쪽으로 한 칸 이동  
            j -= 1  
        a[j + 1] = key # 찾은 삽입 위치에 key를 저장
```

```
d = [2, 4, 5, 1, 3]  
ins_sort(d)  
print(d)
```



Thank You